

Computational Social Science (CSS) Bootcamp

Fall 2026

Duration and Times: September 16th, 17th, 22nd, 23rd, and 24th. All days except the 22nd are from 10 to 12 and 14 to 17. The 22nd is from 9 to 13.

Format: 2h morning session, lunch break, 3h evening session (with the exception of the 22nd, which will be a single, uninterrupted session).

Location: At ENSAE, in room 2020 on the 16th, room 1002 on the 17th, room 2035 on the 22nd, room 1002 on the 23rd, and room 1002 on the 24th.

Teachers: Marshall A. Taylor on the 16th, 22nd, 23rd, and 24th. Emilien Schultz on the 17th.

Teacher contact: marshall.taylor@ensae.fr and emilien.schultz@ensae.fr

1. Objectives

Computational social science requires students to wrangle complex data structures and use statistical methods to find and test patterns in them—all while writing reproducible code in collaborative environments. Because our incoming Master 2 students come from different academic backgrounds (engineers, social sciences, applied math), the goal of this one-week camp is to get everyone on the same page before regular classes start in October. The camp focuses on setting up software, teaching basic coding hygiene, overviewing some statistical basics, and introducing some of the tools and topics students will encounter throughout the year. The material will be hands-on, accessible, and tailored to students' skill levels.

SQD students will also find the bootcamp useful and are highly encouraged to participate.

Note that the specific contents listed below are subject to change.

2. Day-by-Day Sessions

Day	Thematic Focus	Morning: Tools & Theory	Afternoon: Group Application
Day 1	Collaborative Foundations	Unix, Git, GitHub, Python/R IDE setup	Group data wrangling and version control
Day 2	Data Management & Compute	Filesystem/formats, remote infrastructures, deployment, reproducibility, security	Deploying a remote treatment with a separation between code, storage and computing
Day 3	Stats Fundamentals	Linear models and R/Stata basics	Replicating a regression table
Day 4	Text & Dimensionality	Preprocessing, matrix math, basics of dimensionality reduction (SVD)	Mapping word meaning
Day 5	AI Workflows and Bayes	Local LLMs, debugging, Ollama, Opencode, Bayes concepts	Broken pipeline challenge

Day 1: Collaborative Foundations

Morning: Tools & Theory Topics: Unix, Git, GitHub, Python/R IDE setup.

Overview: The focus is on establishing a unified, local coding environment. Students will learn how to initialize and structure reproducible environments in Positron. The session will cover fundamental Git commands to link their local environments to GitHub repositories.

Afternoon: Group Application Challenge: Group data wrangling and version control.

Overview: Students are assigned to groups and tasked with setting up a shared GitHub repository. You will be given a messy data extract from the European Social Survey (ESS) and must divide the cleaning and wrangling tasks using R or Python, navigating and resolving merge conflicts as you build a single, unified data frame.

Day 2: Data Management & Compute

Morning: Tools & Theory Topics: Data Management & Compute & Filesystem/formats ; remote infrastructures ; deployment ; reproducibility.

Overview: This session will address the question of how to organize your project to be able to run smoothly analysis on remote servers with higher reproducibility. Building on collaborative skills, it will introduce the use of Onyxia datalab and remote server with SSH along with HPC notions, with a interest in discussing the best practices for projects organization.

Afternoon: Group Application Challenge: Deploying an analysis on Onyxia.

Overview: Groups will have to run an analysis of a large dataset (Wikipedia or Reddit) remotely using Onyxia and collaborating on Github for the code. The goal will be to maximize the reproducibility of the treatment.

Day 3: Stats Fundamentals

Morning: Tools & Theory Topics: Linear models and R/Stata basics.

Overview: This session introduces the R and Stata interfaces side-by-side, focusing on basic syntax for statistical modeling. We will use this session to cover some fundamentals of statistical theory: probability distributions, point estimates, variance, hypothesis testing, and the OLS estimator.

Afternoon: Group Application Challenge: Replicating a regression table.

Overview: Groups receive a straightforward, published quantitative paper and its raw replication data. You must write a script to clean the data and replicate the primary regression table in Stata. You must then reproduce those results in R.

Day 4: Text & Dimensionality

Morning: Tools & Theory Topics: Preprocessing, matrix math, basics of dimensionality reduction (SVD).

Overview: An introduction to the concept of high-dimensional data using the case of text data. Covers the foundational text-processing pipeline: tokenization, stop-word removal, and building a document-term matrix. The statistical review covers singular value decomposition (SVD)—the bedrock of more advanced forms of dimension reduction.

Afternoon: Group Application Challenge: Mapping Word Meaning.

Overview: Providing a large corpus of unstructured text, groups must collaboratively process the documents and conduct a classic latent semantic analysis (LSA). You will conclude by generating and presenting brief visual summaries of the extracted themes/topics using PCA biplots.

Day 5: AI Workflows & Bayes

Morning: Tools & Theory Topics: Local LLMs, debugging, Ollama, Opencode, Bayes concepts.

Overview: The theoretical portion reviews conditional probability and Bayes' Theorem. The tools component focuses on effectively using LLMs with Ollama and Opencode locally to generate code documentation, interpret complex error messages, and debug scripts safely within Positron.

Afternoon: Group Application Challenge: Broken pipeline challenge.

Overview: Groups receive a complex analytical script riddled with intentional syntax errors, broken paths, and logical bugs. You are forbidden from fixing it manually. You must rely on LLM prompts to diagnose the errors, patch the code, and document the fixes—all using local LLMs within Opencode.